

Justificación del Patrón: Idempotencia

Aquí explicamos el concepto más importante de la solución técnica: la Idempotencia. Mostramos cómo la lógica del código previene errores al validar dinámicamente la existencia de datos y resolver relaciones utilizando atributos únicos en lugar de IDs fijos (hardcodeados), asegurando estabilidad en cualquier entorno.



Seguridad

Un script **idempotente** permite ejecutar el proceso múltiples veces (1 o 100 veces). Si los datos ya existen, el script simplemente los ignora, previniendo errores críticos de clave duplicada que rompen el despliegue.



Independencia de Entorno

Evitamos asumir IDs fijos (ej. Sala ID = 1). En su lugar, buscamos dinámicamente atributos únicos de negocio (como `nombre="Sala A"` o `legajo="E-001"`). Esto asegura que el script funcione correctamente en la máquina local de cualquier desarrollador.

Lógica up() Lógica down()

```
up: async () => {  
  // 1. Verifica (SELECT) por atributo único  
  const [existe] = await sequelize.query(  
    'SELECT * FROM Tabla WHERE nombre = :nombre'  
  );
```